High-Performance E-Commerce Backend Engine

Testing Guide — Requirements 1 through 5

This document walks through the exact commands needed to demonstrate each of the first five non-functional requirements end-to-end. Every step lists the command to run, the output you should expect, and how to interpret it.

Stack: Django 5 + PostgreSQL 16 + Redis 7 + Celery + Nginx, all running under Docker Compose. The host is Windows + Docker Desktop.

Conventions used in this guide

Host shell	PowerShell or Git-Bash on Windows. Examples use Git-Bash syntax.
Container shell	Anything prefixed with docker compose exec ....
BASE_URL	From host: http://localhost:8080. From inside a container: <font face='C
Demo credentials	Username demo / password demo12345 (s
Stack root	C:\Users\ibrah\Desktop\high-performance-ecommerce-engine

0. One-time setup

Skip ahead to Section 1 if the stack is already running.

0.1 Start Docker Desktop

Open the Docker Desktop application from the Start menu and wait until the whale icon in the system tray stops animating. Verify from a shell:

```
docker info | grep "Server Version"
```

You should see a line such as `Server Version: 29.4.3`.

0.2 Build and start the stack

```
cd C:/Users/ibrah/Desktop/high-performance-ecommerce-engine
docker compose build
docker compose up -d
```

First build takes 1–3 minutes (image pulls and pip install). Subsequent starts are seconds.

0.3 Confirm every container is healthy

```
docker compose ps
```

Expected: eight containers — `hpe_postgres` and `hpe_redis` shown as *healthy*; three `hpe_webN` Django app servers; `hpe_nginx`; `hpe_celery_worker`; `hpe_celery_beat`. The migrator container has already exited 0.

0.4 Seed demo data

```
docker compose exec -T web1 python scripts/seed.py
```

Expected output:

```
created user: demo / demo12345 (staff)
created: LAPTOP-001 stock=100
created: PHONE-001  stock=100
created: RACE-001   stock=50
created: BOOK-001   stock=200
seed done.
```

0.5 Health probe through the load balancer

```
curl -s -D - -o /dev/null http://localhost:8080/api/health/
```

Expected headers include:

```
HTTP/1.1 200 OK
X-Instance: web1   (or web2 / web3)
X-Served-By: 172.x.x.x:8000
X-Response-Time-Ms: 25.3
```

What this proves. Nginx is routing through to one of three Django workers, the AOP timing middleware is stamping a response-time header on every request, and the database is reachable (the health view runs a real `SELECT 1`).

Requirement 1 — Concurrent Access & Data Integrity

Goal: prove a Race Condition exists on a naive implementation, then prove our pessimistic row-lock implementation makes it impossible.

1.1 Inspect the safe-vs-unsafe code paths

Both checkout endpoints are wired up. The unsafe path is kept on purpose so we can demonstrate the bug.

Safe path	POST /api/orders/checkout/ — pessimistic row lock via <font face='Cou
Demo path	POST /api/orders/checkout-direct/ with body field {'
Source	apps/orders/services.py (safe) and <font face='Courier'>apps/orders/vie

1.2 Run the empirical demo

The script logs in as `demo`, resets `RACE-001` to 50 units, fires 100 concurrent single-unit purchases on the unsafe path, then repeats on the safe path.

```
docker compose exec -T -e BASE_URL=http://nginx web1 \
    python scripts/race_condition_demo.py
```

1.3 What you should see

Phase 1 — UNSAFE: oversell. More than 50 sales succeed (often all 100), final stock is negative or otherwise inconsistent. Example from this machine:

```
=== Phase 1: UNSAFE checkout (proving the race exists) ===
  mode                            UNSAFE (no lock)
  requests_fired                  100
  successful_sales (HTTP 201)     100
  out_of_stock_rejected (HTTP 409)  0
  stock_before                    50
  stock_after                     38
  consistent                      False
```

Phase 2 — SAFE: exactly 50 sales, exactly 50 conflicts:

```
=== Phase 2: SAFE checkout (with row lock) ===
  mode                            SAFE (SELECT FOR UPDATE)
  successful_sales (HTTP 201)     50
  out_of_stock_rejected (HTTP 409)  50
  stock_after                     0
  consistent                      True
```

Interpretation. Phase 1's `consistent=False` with `successes + stock_after != 50` is the textbook Race-Condition fingerprint. Phase 2's outcome is the *only* arithmetically valid result for 100 buyers chasing 50 units — guaranteed by the row lock that holds for the duration of every checkout transaction.

1.4 See the lock in action from Postgres

Optional: in one shell open psql and watch the locks while the demo runs in another.

```
docker compose exec postgres psql -U ecommerce -d ecommerce \
  -c "SELECT relation::regclass, mode, granted FROM pg_locks \
      WHERE relation IS NOT NULL ORDER BY pid;"
```

During Phase 2 you will see `RowExclusiveLock` and `ExclusiveLock` entries on `catalog_product` blink in and out.

Requirement 2 — Resource Management & Capacity Control

Goal: prove the system has a hard ceiling on simultaneous heavy requests and sheds excess load with `HTTP 503` rather than thrashing.

2.1 What is in place

Per-process cap	BoundedSemaphore(MAX_CONCURRENT_HEAVY_REQUESTS) in <
Gunicorn	--workers 3 --threads 4 --max-requests 1000 per web container
Celery worker	--concurrency=4 --max-tasks-per-child=500
DB pool	CONN_MAX_AGE=60 in settings.py

2.2 Lower the cap to make the limit visible

Edit `.env` in the project root, change the line:

```
MAX_CONCURRENT_HEAVY_REQUESTS=2
```

Restart the three app servers (Compose picks up the new env on recreate):

```
docker compose up -d --force-recreate web1 web2 web3
```

2.3 Trigger backpressure

Re-run the race-condition demo. With the cap at 2 per process, the bursts of 100 concurrent checkouts will overshoot the semaphore and the middleware will return `HTTP 503` for the excess requests.

```
docker compose exec -T -e BASE_URL=http://nginx web1 \
    python scripts/race_condition_demo.py
```

You should see a non-zero `other_errors` count in the demo summary. Confirm in the access log:

```
docker compose logs web1 web2 web3 2>&amp;1 | grep " 503 " | head
```

Sample line:

```
hpe_web2 | [17/May/2026:08:58:11 +0000] "POST /api/orders/checkout-direct/" 503 56
```

2.4 Confirm the AOP-style timing middleware is logging

```
docker compose logs web1 2>&amp;1 | grep -E "slow request|X-Response-Time" | head
```

Sample line:

```
apps.core inst=web1 :: POST /api/orders/checkout-direct/ 142.6ms
```

Interpretation. 503 responses are intentional and good. They are the system saying "I am at capacity — try a sibling worker or back off," rather than queuing forever and dragging the whole pipeline down. Nginx's `max_fails=3 fail_timeout=10s` then routes new traffic to the two healthy peers — Req-2 and Req-5 form a closed control loop.

2.5 Restore the cap

Set `MAX_CONCURRENT_HEAVY_REQUESTS=8` in `.env` and recreate the web containers.

```
docker compose up -d --force-recreate web1 web2 web3
```

Requirement 3 — Asynchronous Queues

Goal: prove that checkout returns to the user the instant the row lock is released, with heavy work (invoice rendering, notifications) executed afterwards on a Celery worker over a Redis broker.

3.1 What is queued vs synchronous

Synchronous	Stock decrement, order insert, response to client
Asynchronous	send_invoice_email (300–600 ms) and send_order
Source	Dispatch: apps/orders/views.py; tasks: <font face='Courier'>apps/orders

3.2 Stream the worker log in one terminal

```
docker compose logs -f celery_worker
```

3.3 In a second terminal, log in and fire a checkout

```
# get a token
TOKEN=$(curl -s -X POST http://localhost:8080/api/accounts/login/ \
         -H 'Content-Type: application/json' \
         -d '{"username":"demo","password":"demo12345"}' \
         | python -c "import sys,json;print(json.load(sys.stdin)['token'])")

# direct checkout — 1 unit of LAPTOP-001 (id may differ; check
# GET /api/catalog/products/)
curl -s -X POST http://localhost:8080/api/orders/checkout-direct/ \
    -H "Authorization: Token $TOKEN" \
    -H 'Content-Type: application/json' \
    -d '{"items":[{"product_id":1,"quantity":1}],"unsafe":false}'
```

3.4 What you should see

The HTTP response comes back in <100 ms with the order JSON. *After* the response, the worker terminal logs something like:

```
Task apps.orders.tasks.send_invoice_email[…] received
timed[task.send_invoice_email] 313.25ms
Task apps.orders.tasks.send_invoice_email[…] succeeded in 0.319s
Task apps.orders.tasks.send_order_notifications[…] received
timed[task.send_order_notifications] 133.48ms
```

Interpretation. The `timed[…]` lines are emitted by the AOP decorator `@timed` applied to each Celery task. The fact that they appear in the *worker* log, not in the web log, is what proves the work was off-loaded from the request path.

3.5 Confirm acks-late + bounded prefetch

```
docker compose exec -T web1 python -c \
  "from django.conf import settings; \
    print('acks_late=', settings.CELERY_TASK_ACKS_LATE); \
    print('prefetch=', settings.CELERY_WORKER_PREFETCH_MULTIPLIER)"
```

Expected: `acks_late= True` and `prefetch= 1`. Together they guarantee (a) a crashed worker triggers re-delivery and (b) no worker hoards messages while peers idle.

Requirement 4 — Batch Processing

Goal: prove a background job rolls up a full day's order items in fixed-size chunks (server-side cursor), not by buffering the whole result set.

4.1 What runs and when

Task	apps.orders.tasks.rollup_daily_sales
Schedule	Celery Beat — cron 00:05 UTC daily
Strategy	queryset.iterator(chunk_size=500) — Postgres server-side cursor
Output	orders_dailysalesreport table, one row per date

4.2 Fire the job on demand (no need to wait for 00:05 UTC)

```
curl -s -X POST http://localhost:8080/api/orders/reports/trigger/ \
    -H 'Content-Type: application/json' \
    -d '{"date":"2026-05-17"}'
```

Expected:

```
{"task_id":"<uuid>","queued":true}
```

4.3 Watch it execute

```
docker compose logs celery_worker 2>&amp;1 \
  | grep -E "rollup_daily_sales|rollup done" | tail
```

Expected (numbers depend on how many orders you have on that date):

```
Task apps.orders.tasks.rollup_daily_sales[…] received
rollup done {'date': '2026-05-17', 'orders': 150, 'units': 150,
             'revenue': '1500.00', 'chunks_processed': 1,
             'chunk_size': 500}
timed[task.rollup_daily_sales] 49.52ms
```

4.4 Read the persisted result

```
curl -s http://localhost:8080/api/orders/reports/daily/
```

Expected:

```
[{"id":1,"date":"2026-05-17","orders_count":150,
  "units_sold":150,"gross_revenue":"1500.00",
  "generated_at":"2026-05-17T08:44:36.260815Z"}]
```

4.5 Generate a bigger dataset to see real chunking

Once you have enough orders to exceed `chunk_size=500`, the `chunks_processed` field in the rollup log line will go above 1 — that is the proof the iterator is streaming rather than loading the whole result set.

> **Interpretation.** Postgres FETCHes the next 500 rows only when the previous chunk has been consumed. Python peak memory therefore stays O(500) regardless of whether the day saw 5 000 or 5 000 000 line items. Single `.aggregate(Sum(...))` calls would also finish quickly on small data but would not scale the same way, and would not exhibit the "process in chunks" behaviour the rubric asks for.

Requirement 5 — Load Distribution

Goal: prove that requests fan out across three identical Django app servers behind Nginx, and that the `least_conn` algorithm produces a sensible distribution under concurrent load.

5.1 What is in place

Topology	Three identical app servers web1 / web2 / web3 behind <font face='Cou
Algorithm	least_conn in nginx/nginx.conf
Observability	X-Served-By (Nginx) and X-Instance (Djang
Failure handling	max_fails=3 fail_timeout=10s per upstream

5.2 See it sequentially (expected: mostly one server)

```
for i in $(seq 1 9); do
  curl -s -D - -o /dev/null http://localhost:8080/api/health/ \
    | grep -E '^X-Instance'
done
```

Sequential requests close their connection before the next starts, so each request finds all three upstreams at zero active connections; Nginx's tie-break sends them all to the first listed server. This is the *correct* behaviour for `least_conn` under sequential traffic, not a bug.

5.3 See it concurrently (expected: fan out)

```
for i in $(seq 1 30); do
  ( curl -s -D - -o /dev/null http://localhost:8080/api/health/ \
      | grep -E '^X-Instance' &amp;amp; )
done; wait
```

Expected: a mix of all three workers, roughly even. Real output from this machine:

```
X-Instance: web1   x 11
X-Instance: web2   x 10
X-Instance: web3   x 9
```

5.4 See the upstream Nginx routed to

```
curl -s -D - -o /dev/null http://localhost:8080/api/health/ \
  | grep -E 'X-Instance|X-Served-By'
```

Expected:

```
X-Instance: web2
X-Served-By: 172.18.0.4:8000
```

5.5 Verify failover takes a sick worker out of rotation

Stop one worker, hit the LB repeatedly, then bring it back:

```
docker compose stop web2
for i in $(seq 1 12); do
  curl -s -D - -o /dev/null http://localhost:8080/api/health/ \
    | grep -E '^X-Instance'
done
docker compose start web2
```

Expected: while web2 is down, every response shows `X-Instance: web1` or `X-Instance: web3`. No 502s.

> **Interpretation.** `least_conn` dominates `round_robin` for e-commerce because request cost is non-uniform: product-list calls finish in 5 ms, checkout calls take 50–200 ms. Round-robin would happily route a third checkout to a worker that already has two pending, while peers idle. `least_conn` always picks

the upstream with the fewest active connections, evening out tail latency under bursty load.

Appendix — Troubleshooting

"Connection refused" when running scripts inside a container

Inside any container, `localhost:8080` points back at the container itself, not at Nginx. Use `http://nginx` instead and pass it as an env var: `docker compose exec -T -e BASE_URL=http://nginx web1 ...`

Migrator container failed

```
docker compose logs migrator | tail
```

Re-run with `docker compose up -d --force-recreate migrator`.

Stale state — start clean

```
docker compose down -v   # also drops the Postgres + Redis volumes
docker compose up -d
docker compose exec -T web1 python scripts/seed.py
```

Where to find each requirement in the code

Req 1	apps/orders/services.py (lock), apps/orders/views.p
Req 2	apps/core/middleware.py::CapacityControlMiddleware, gunicorn args in
Req 3	apps/orders/tasks.py, dispatch in apps/orders/view
Req 4	apps.orders.tasks.rollup_daily_sales, beat schedule in <font face='Cour
Req 5	nginx/nginx.conf, instance tagging in apps/core/mid
AOP	apps/core/aop.py (decorator), <font face='Courier'>apps/core/middlewar